Graficación por Computadora
Tarea 5

Francisco Alejandro Arganis Ramírez

24 de marzo de 2026

1 Descripción

El programa carga el modelo del archivo `Cube_Quads.obj` usando la biblioteca `tinyobjloader`. El modelo es un cubo descrito por 6 caras cuadradas que son trianguladas de forma automática por `tinyobjloader`.

Como `Cube_Quads.obj` solo contiene información de vértices, después de cargar el modelo se calculan manualmente las normales por triángulo. Adicionalmente, se asigna un color aleatorio a cada par de triángulos consecutivos, con lo cual cada cara del cubo recibe su propio color.

Una vez construido el buffer de vértices, se renderiza una escena que consiste en:

- El cubo cargado desde `Cube_Quads.obj`. El material del cubo tiene coeficientes ambiental $(0.3, 0.3, 0.3)$, coeficiente difuso $(0.07568, 0.61424, 0.07568)$, coeficiente especular $(0.633, 0.727811, 0.633)$ y valor de brillo especular 76.8.
- Dos luces puntuales sin atenuación por distancia. Una luz blanca se encuentra en $(70, 50, 100)$ con componente ambiental $(0.1, 0.1, 0.1)$, componente difusa $(0.8, 0.8, 0.8)$ y componente especular $(0.8, 0.8, 0.8)$. Una luz roja se encuentra en $(70, -50, 100)$ con componente ambiental $(0.1, 0.1, 0.1)$, componente difusa $(0.8, 0, 0)$ y componente especular $(0.8, 0, 0)$. La componente ambiental global de la escena es $(0.1, 0.1, 0.1)$.
- Dos cámaras en las posiciones $(40, 60, 50)$ y $(-20, -40, 60)$ viendo hacia al origen.

En el fragment shader se implementa el modelo de reflexión Blinn–Phong, sustituyendo $R \cdot V$ por $H \cdot N$ para el cálculo de la reflexión especular, donde:

- N es la dirección unitaria de la normal a la superficie en la posición del fragmento.
- V es la dirección unitaria desde la posición del fragmento hacia la cámara.
- L es la dirección unitaria desde la posición del framgento hacia la luz.
- R es el vector resultante de reflejar L con respecto a la normal.
- H es la dirección unitaria a la mitad entre L y V, es decir, $H = \frac{L+V}{||L+V||}$.

A la imagen renderizada se le aplica un filtro de convolución con el siguiente kernel de 3×3 antes de mostrarla en la ventana

$$\begin{pmatrix} -\frac{1}{9} & -\frac{1}{9} & -\frac{1}{9} \\ -\frac{1}{9} & \frac{17}{9} & -\frac{1}{9} \\ -\frac{1}{9} & -\frac{1}{9} & -\frac{1}{9} \end{pmatrix}$$

El mismo kernel se aplica a los 3 canales de la imagen.

La aplicación provee controles en la interfaz gráfica para seleccionar la cámara y el tipo de proyección, modificar la transformación y material del cubo, prender o apagar las luces, modificar los valores de las componentes de las luces, aplicar o no el filtro y modificar los valores del kernel.

El código está estructurado de la siguiente manera:

- `gmath.cpp`. Contiene funciones para la manipulación de vectores y matrices.
- `dip.cpp`. Contiene funciones para el procesamiento digital de imágnes. En particular, contiene la implementación del filtro de convolución.
- `app.cpp`. Define las variables usadas por la aplicación, incluyendo el material del cubo, las posiciones de las cámaras, las luces de la escena y los valores del kernel. También contiene las funciones `vertex_shader` y `fragment_shader` que se pasan a la línea de renderizado y `load_obj` que es donde se carga el modelo del cubo.
- `pipeline.cpp`. Contiene la implementación de la línea de renderizado.
- `gui.cpp`. Contiene la implementación de las estructuras que heredan de `QWidget` usadas para mostrar la ventana y los controles de la interfaz.
- `main.cpp`. Inicializa la ventana e inicia la aplicación.
- `tiny_obj_loader.h`. Es la biblioteca usada para cargar archivos obj.

2 Compilar y ejecutar

Para compilar el programa, en Linux, abrir una terminal en la dirección donde se encuentran `src`, `include` y `tiny_obj_loader.h` y ejecutar

```
g++ src/*.cpp -I . -I include $(pkg-config --cflags Qt6Core
   Qt6Widgets) $(pkg-config --libs Qt6Core Qt6Widgets)
```

Para iniciar el programa ya compilado, con el archivo `Cube_Quads.obj` en la misma dirección que el ejecutable `a.out`, ejecutar

```
./a.out
```

Se requiere:

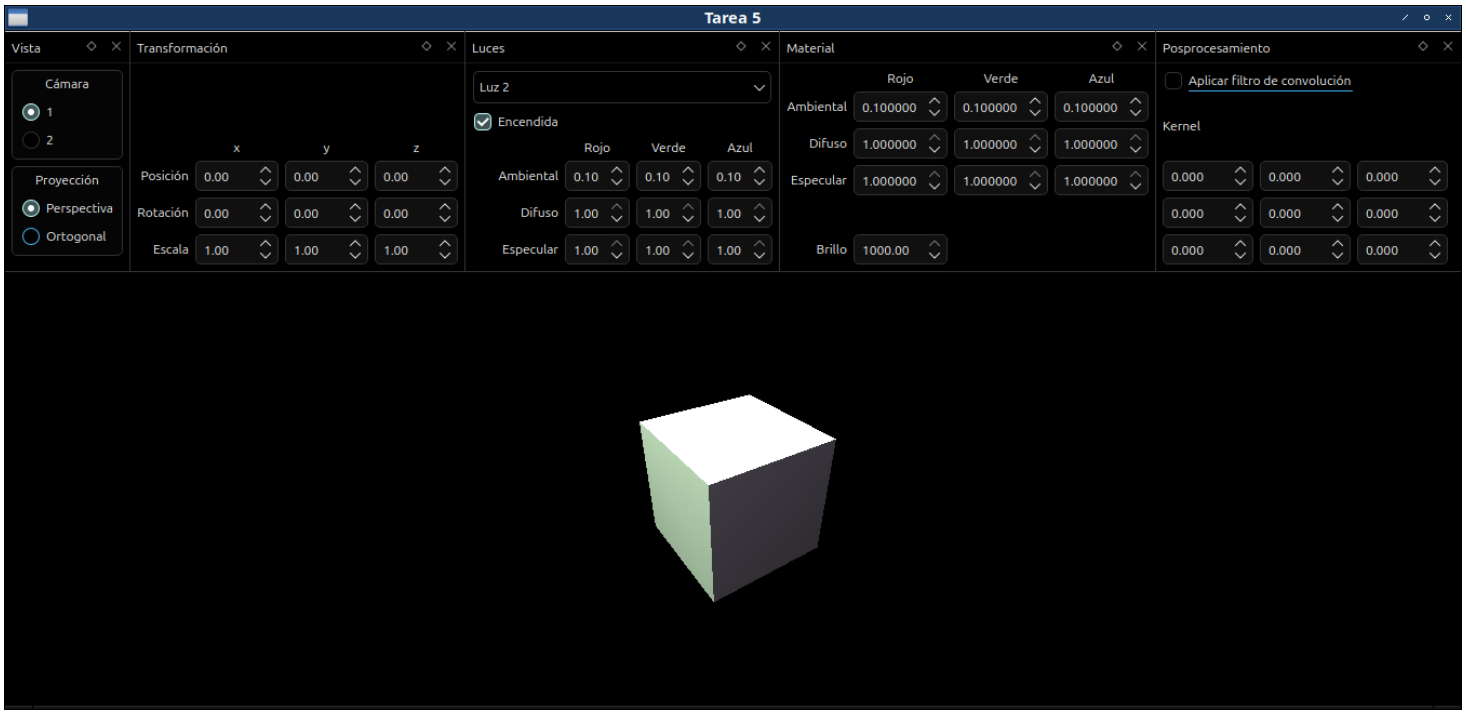
- `gcc` 15.2.0 o superior
- `qt` 6.9.2 o superior

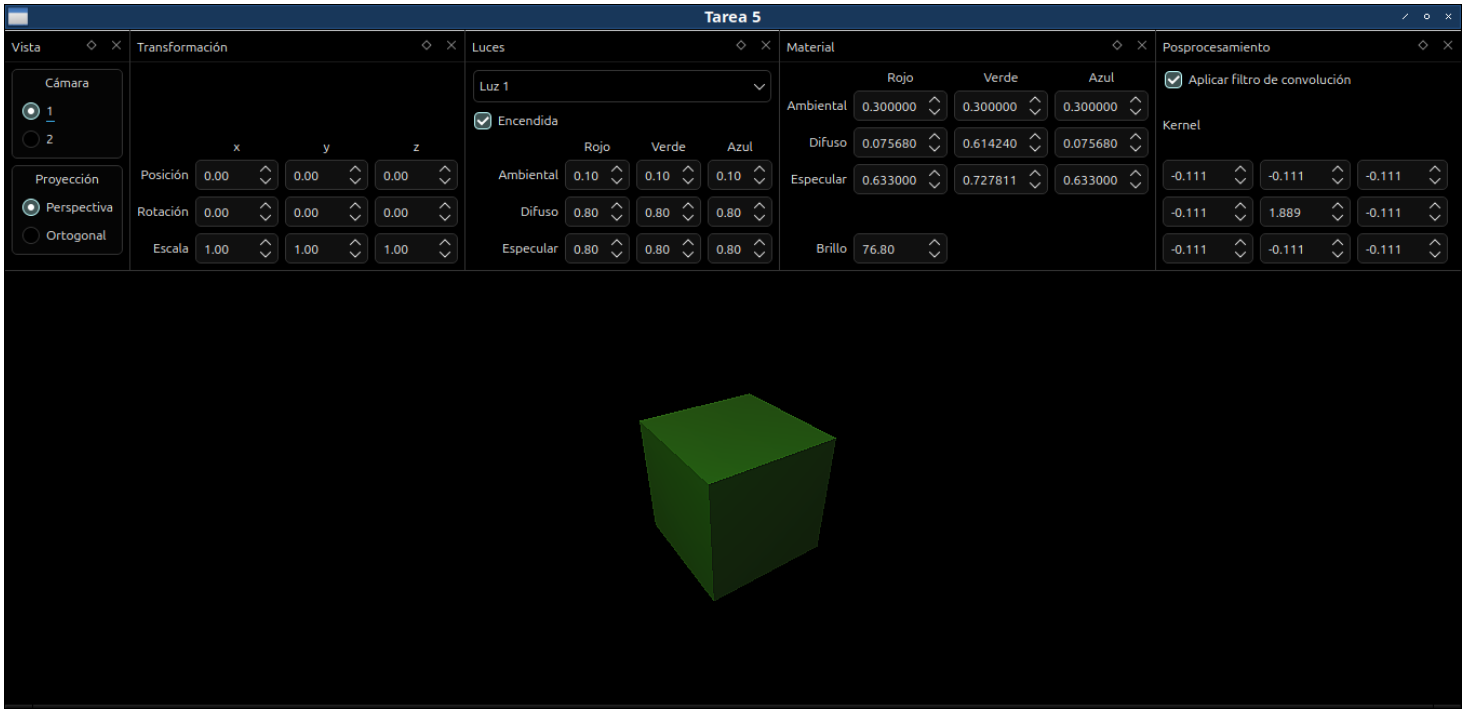
- `pkg-config` 1.8.1 o superior

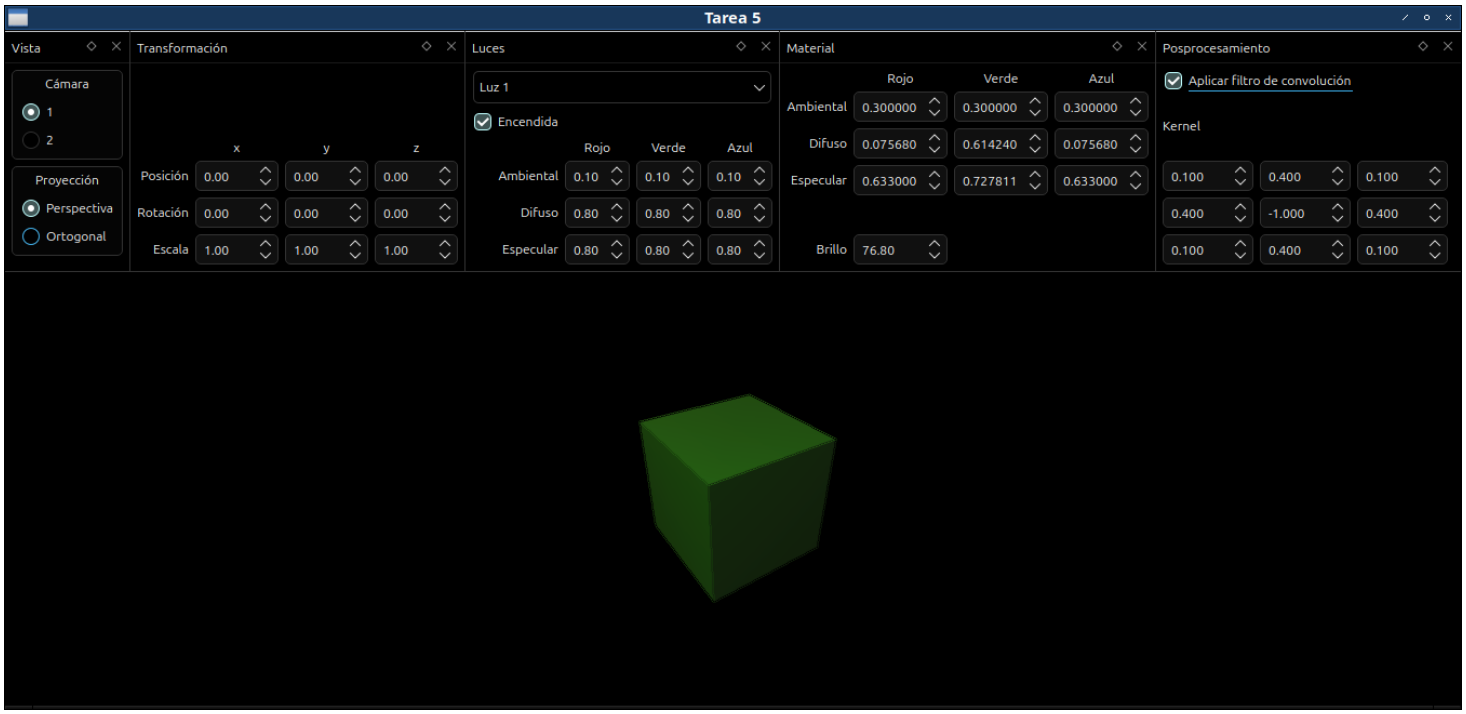
3 Repositorio

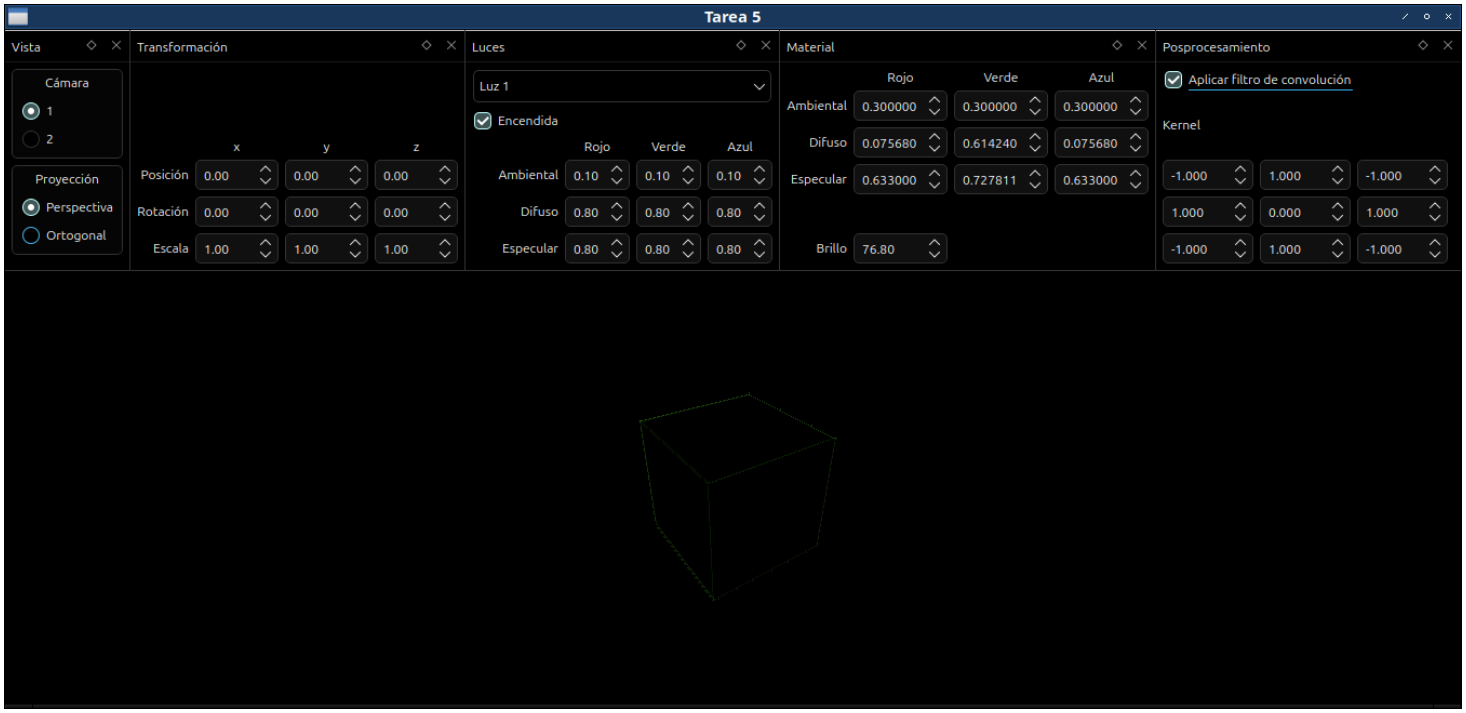
`https://github.com/FranciscoAlejandroArganis/Graficacion_por_Computadora_2026-2`

4 Capturas de pantalla




Tarea 5
Vista
Cámara
1
2
Proyección
Perspectiva
Ortogonal
Transformación
x y z
Posición 0.00 0.00 0.00
Rotación 0.00 0.00 0.00
Escala 1.00 1.00 1.00
Luces
Luz 1
Encendida
Rojo Verde Azul
Ambiental 0.10 0.10 0.10
Difuso 0.80 0.80 0.80
Especular 0.80 0.80 0.80
Material
Rojo Verde Azul
Ambiental 0.300000 0.300000 0.300000
Difuso 0.075680 0.614240 0.075680
Especular 0.633000 0.727811 0.633000
Brillo 76.80
Posprocesamiento
Aplicar filtro de convolución
Kernel
-0.111 -0.111 -0.111
-0.111 1.889 -0.111
-0.111 -0.111 -0.111


Tarea 5
Vista
Cámara
1
2
Proyección
Perspectiva
Ortogonal
Transformación
x y z
Posición 0.00 0.00 0.00
Rotación 0.00 0.00 0.00
Escala 1.00 1.00 1.00
Luces
Luz 1
Encendida
Rojo Verde Azul
Ambiental 0.10 0.10 0.10
Difuso 0.80 0.80 0.80
Especular 0.80 0.80 0.80
Material
Rojo Verde Azul
Ambiental 0.300000 0.300000 0.300000
Difuso 0.075680 0.614240 0.075680
Especular 0.633000 0.727811 0.633000
Brillo 76.80
Posprocesamiento
Aplicar filtro de convolución
Kernel
0.100 0.400 0.100
0.400 -1.000 0.400
0.100 0.400 0.100


Tarea 5
Vista
Cámara
1
2
Proyección
Perspectiva
Ortogonal
Transformación
x
y
z
Posición 0.00 0.00 0.00
Rotación 0.00 0.00 0.00
Escala 1.00 1.00 1.00
Luces
Luz 1
Encendida
Rojo
Verde
Azul
Ambiental 0.10 0.10 0.10
Difuso 0.80 0.80 0.80
Especular 0.80 0.80 0.80
Material
Rojo
Verde
Azul
Ambiental 0.300000 0.300000 0.300000
Difuso 0.075680 0.614240 0.075680
Especular 0.633000 0.727811 0.633000
Brillo 76.80
Posprocesamiento
Aplicar filtro de convolución
Kernel
-1.000 1.000 -1.000
1.000 0.000 1.000
-1.000 1.000 -1.000